DELAYBENCH: Evaluating Reinforcement Learning under Delayed Observations and Actions

Francisco M. López **Lukas Henssler** **Jochen Triesch**
Frankfurt Institute for Advanced Studies
lopez@fias.uni-frankfurt.de

Abstract

Reinforcement learning (RL) algorithms are typically trained and tested under the idealized assumption of instantaneous interactions, where observations reflect the current state of the environment and actions are executed immediately. However, biological and robotic agents are subject to significant sensorimotor delays. Despite growing interest in delay-aware methods, there is no standardized framework for evaluating how such delays impact RL performance. We introduce DELAYBENCH, a benchmark for evaluating RL under delayed observations and actions. DELAYBENCH can be applied to existing environments, enabling controlled experiments across a range of observation and action delay configurations. Using DELAYBENCH, we conduct a systematic study of RL algorithms over multiple environments. We benchmark algorithms' robustness across increasing delays. We find that standard methods degrade rapidly under modest delays, whereas delay-aware methods (history stacking, recurrent architectures, and forward models) improve robustness. DELAYBENCH establishes a systematic approach for benchmarking RL in the presence of sensorimotor delays and facilitates the development of new delay-aware algorithms.

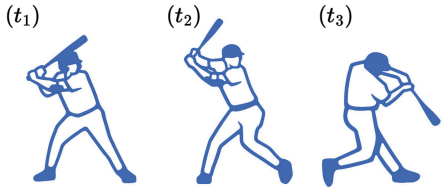
Code: https://github.com/fcomlop/delaybench/tree/anonymous

1 Introduction

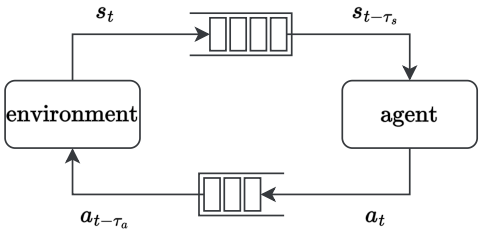
Reinforcement learning (RL) agents learn behaviors through trial-and-error while interacting with their environments. RL algorithms are typically trained and tested under the idealized assumption of instantaneous interactions [1], where observations reflect the current state of the environment and actions are executed immediately. This assumption rarely holds in real-world agents. Both biological and robotic agents operate under significant sensorimotor delays [2, 3, 4], which accumulate across the sensing, decision-making, and actuating stages. These delays necessarily alter the information available to the agent and limit its ability to predict the consequences of its interactions with the environment (see Fig. 1). Under sensorimotor delays, the learning problem becomes temporally misaligned. Despite their importance for real-world problems, sensorimotor delays are not systematically evaluated in RL. Existing protocols typically focus on comparisons of algorithms in the context of instantaneous interactions. Consequently, it is unclear how robust different algorithms are in the face of sensorimotor delays. Likewise, delay-aware algorithms that explicitly aim to compensate for such delays are evaluated in custom-made environments [5, 6, 7, 8, 9, 10, 11] (see [12] for a recent review). The lack of a standardized framework for evaluating how sensorimotor delays impact RL performance impedes the comparison of state-of-the-art methods and the development of novel approaches. Appendix A provides a detailed overview of related works, including studies of sensorimotor delays in RL and real-world systems.

In this work, we introduce DELAYBENCH, a benchmark for evaluating RL under controlled sensorimotor delays. DELAYBENCH extends general-purpose RL environments with separate observation

Preprint.



(a) Example of real-world delay problem



(b) RL perception-action loop with delays

Figure 1: Modeling sensorimotor delays. **(a)** A classic example of sensorimotor delays for a real-world biological agent. A baseball batter observes the pitcher's movement at time t_1, decides to swing his bat at time t_2, and this movement is executed at t_3. Therefore, the decision at t_2 is made from the observation at t_1 but must anticipate the position of the ball at t_3. **(b)** To model these sensorimotor delays, we extend the classic RL perception-action loop by means of first-in, first-out (FIFO) buffers for the actions and observations.

and action delay mechanisms, transforming each environment into a collection of tasks parametrized by the delays. This design enables the evaluation of algorithms by means of return-delay curves, which describe how performance changes with increasing temporal misalignment. To facilitate comparison, we summarize the robustness against delays using a single metric based on the area under the return-delay curve. This score simultaneously quantifies performance in the zero-delay condition and robustness under increasing delays.

Our contributions are as follows:

- we develop a collection of delay-augmented RL environments with a standardized interface that can be used with most existing RL libraries,
- we provide an evaluation protocol to score robustness under observation and action delays,
- we perform an empirical study of popular RL algorithms as well as three delay-aware extensions: history stacking, recurrent architectures, and forward models.

Our results reveal that common RL algorithms degrade rapidly under modest delays, whereas delay-aware methods can alleviate the degradation. We also find that delay robustness is only weakly related to zero-delay performance, thus demonstrating the importance of considering sensorimotor delays as a distinct axis for evaluating RL algorithms. We release DELAYBENCH as an open-source platform to support future research on the effects of delays on RL.

2 DELAYBENCH

At its core, DELAYBENCH consists of a collection of RL environments with controlled observation and action delays as well as a standardized evaluation protocol summarized into a single metric for quantifying robustness.

2.1 Formalism

Instantaneous interactions. RL algorithms typically assume a Markov decision process (MDP) [1] given by

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma), \tag{1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, p is the state transition probability distribution, r is the reward function, and $\gamma \in [0, 1)$ is the discount factor. Time runs in discrete steps $t \in \{0, 1, 2, \ldots\}$.

Under the Markov property, the current state s_t is sufficient to predict the distribution of future states and rewards, given the current action. That is, the interactions between the agent and its environment result in state updates

$$s_{t+1} \sim p(\cdot \mid s_t, a_t) \ , \tag{2}$$

where the actions are selected by the agent from the current state following a policy π:

$$a_t \sim \pi(\cdot \mid s_t) \ . \tag{3}$$

We note that a more general formulation of the RL problem is as a partially observable Markov decision process (POMDP), where the agent receives an observation o_t that does not fully determine the underlying state s_t. Such partial observability adds an additional challenge that is orthogonal to the problem of delays. We focus on the classic MDP setting here mostly for didactic reasons, but DELAYBENCH also permits studying the effects of delays in the more general POMDP setting.

Observation and action delays. The introduction of sensorimotor delays requires a relaxation of the MDP because the interactions between the agent and its environment are no longer instantaneous. Concretely, let $\tau_s \in \mathbb{N}$ denote the observation delay and $\tau_a \in \mathbb{N}$ denote the action delay. At the time t, the agent chooses its action a_t using a delayed observation of the state, such that

$$a_t \sim \pi(\cdot \mid s_{t-\tau_s}) \ . \tag{4}$$

That command is not executed immediately. Instead, the agent executes an action it had chosen previously, so that the environment is updated according to

$$s_{t+1} \sim p(\cdot \mid s_t, a_{t-\tau_a}) \ . \tag{5}$$

Observation delays affect the information available to the agent's policy, whereas action delays affect the environment's update. As a result, sensorimotor delays make the process non-Markovian. The information available to the agent is no longer sufficient to determine future states or rewards. Exact Markovianity can only be recovered by expanding the agent's information with the missing observations and actions. Recent efforts in delay-aware RL algorithms attempt to overcome the non-Markovianity by means of approximate predictions of the interactions between the agent and its environment. DELAYBENCH provides a systematic platform to evaluate these algorithms.

2.2 Delay-augmented environments

We construct a collection of delay-augmented environments $\{\mathcal{E}_{\tau_s,\tau_a}\}$ parametrized by discrete observation delays $\tau_s \in \mathbb{N}$ and action delays $\tau_a \in \mathbb{N}$, such that the original environment $\mathcal{E} = \mathcal{E}_{0,0}$ is an MDP. Delays are implemented using first-in, first-out (FIFO) buffers for the actions and observations (see Fig. 1). At reset, the observation FIFO buffer is filled with the first observation returned by the MDP, which is observed by the agent for the first $\tau_s + 1$ steps. The action FIFO buffer is filled with actions uniformly sampled from the action space to avoid systematic biases. All other components of the original environments, such as the reward functions and early termination conditions, are kept untouched.

DELAYBENCH extends a wide range of standard RL environments from the Gymnasium library [13] while retaining the original API. The original environments cover both classic discrete control tasks and continuous control tasks. This selection ensures diversity in environment dynamics, reward structure, and control requirements, which enables a comprehensive assessment of the robustness of RL algorithms to sensorimotor delays in a variety of scenarios. The full list of environments is shown in Table 1. A comprehensive description of the environments, including their observation and action spaces, is provided in Appendix C.2.

2.3 Evaluation protocol

A successful delay-aware algorithm should learn successful policies for large observation and action delays without incurring a significant loss in performance for the zero-delay condition. We propose a benchmarking protocol that captures this intuition and can be followed to train and evaluate each algorithm separately.

Delays. Sensorimotor delays typically entail a degradation in the RL performance, which is upper-bounded by the zero-delay performance and lower-bounded by an infinite delay condition. DELAYBENCH can be used with any set of discrete observation and/or action delays, which users can select

to investigate specific conditions or failure modes. For the default benchmarking, all environments are implemented with a common set of observation and action delays:

$$\tau_s,\, \tau_a \in \{0, 1, 2, 4, 8, 16, T_{\mathcal{E}}\} \,, \tag{6}$$

where $T_{\mathcal{E}}$ is the default maximum episode duration of the original Gymnasium environment $\mathcal{E}$. In practice, the performance for a delay of $T_{\mathcal{E}}$ is measured from a random policy uniformly sampled from the action space and does not require training. This set spans near-instantaneous conditions to severe temporal mismatches. Observation and action delays are applied separately to inquire about their separate effects, but they can also be used simultaneously (see Fig. 2).

Training. For each pair of delays $\{\tau_s, \tau_a\}$, any particular algorithm is used to train a policy π on the delay-augmented environment $\mathcal{E}_{\tau_s,\tau_a}$. In our implementations, we use a fixed training budget consisting of a maximum total number of training steps. However, this is not a necessary requirement and can be omitted, namely for algorithms requiring large amounts of training data to learn internal models, e.g. [8].

After training, the deterministic policy is used to obtain the total returns G_{τ_s,τ_a} over M evaluation episodes . Each experiment is repeated across N random seeds and the results are aggregated across the different seeds. We then compute the average evaluation return

$$\mathcal{G}_{\tau_s,\tau_a} = \frac{1}{NM} \sum_{i=0}^{N} \sum_{i=0}^{M} G^{(i,j)}_{\tau_s,\tau_a} \,. \tag{7}$$

By default, DELAYBENCH uses $N = 20$ random seeds for training and $M = 20$ evaluation episodes after training, yielding a total of 400 values used to compute the average evaluation return.

Return-delay curve. To measure robustness to observation and action delays, we compute return-delay curves (RDCs) from the average evaluation returns. Concretely, the RDC is a decaying sigmoid function of a delay τ given by

$$J(\tau) = J_\infty + \frac{J_0 - J_\infty}{1 + e^{\alpha(\tau - \tau^*)}} \,, \tag{8}$$

where J_0 and J_∞ are the upper and lower asymptotes respectively, α is the slope and τ^* is the inflection point.

We fit separate sigmoids for the observation and action delays, setting the other delay to zero. Additionally, we use the zero-delay condition for the upper asymptote and the $T_{\mathcal{E}}$ delay condition (random policy) for the lower asymptote. If an algorithm's returns drop below this random policy performance in any configuration, in particular due to large delays, this is not penalized by DELAYBENCH. Consequently, each RDC has only 2 free parameters, the slope and the inflection point, that describe the degradation of the performance to either observation or action delays:

$$J_s(\tau) = \mathcal{G}_{T_{\mathcal{E}},0} + \frac{\mathcal{G}_{0,0} - \mathcal{G}_{T_{\mathcal{E}},0}}{1 + e^{\alpha(\tau_s - \tau_s^*)}} \,, \qquad J_a(\tau) = \mathcal{G}_{0,T_{\mathcal{E}}} + \frac{\mathcal{G}_{0,0} - \mathcal{G}_{0,T_{\mathcal{E}}}}{1 + e^{\alpha(\tau_a - \tau_a^*)}} \,. \tag{9}$$

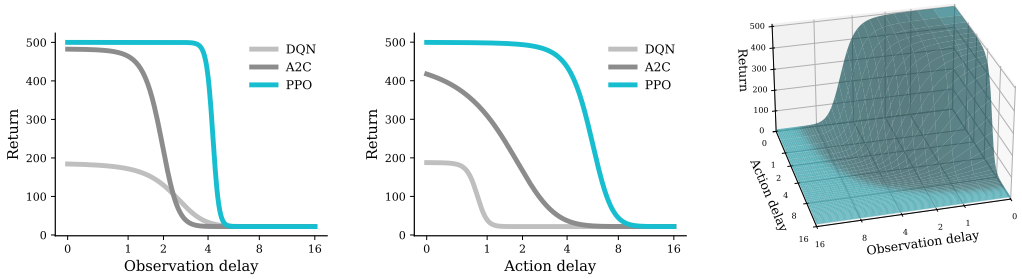
AUC score. Finally, to summarize the resulting RDCs into single metrics, we calculate their area under the curve (AUC) scores:

$$\mathrm{AUC}_{\mathrm{obs}} = \int_0^{T_{\mathcal{E}}} J_s(\tau) - \mathcal{G}_{T_{\mathcal{E}},0}\, d\tau \,, \qquad \mathrm{AUC}_{\mathrm{act}} = \int_0^{T_{\mathcal{E}}} J_a(\tau) - \mathcal{G}_{0,T_{\mathcal{E}}}\, d\tau \,. \tag{10}$$

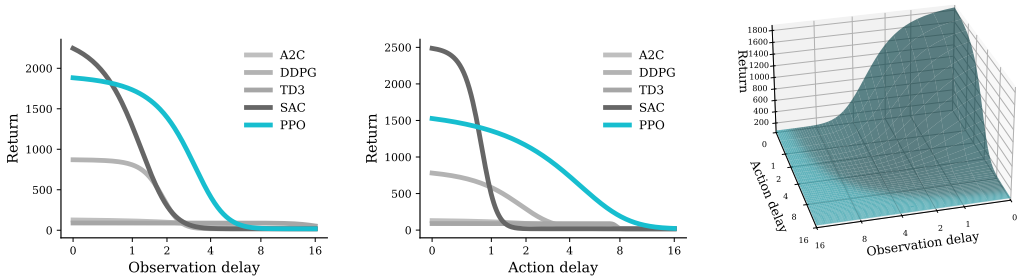
Since the lower asymptote of the RDC corresponds to the random policy returns, it does not depend on the RL algorithm. Therefore, higher AUC scores indicate an algorithm satisfying the two desired properties: (i) high returns for zero-delay condition $\mathcal{G}_{0,0}$, and (ii) slow degradation of this upper asymptote performance, particularly with large inflection points τ_s^* and τ_a^*.

3 Algorithms

To validate DELAYBENCH, we compare the RDCs and their resulting AUC scores for several algorithms across multiple delay-augmented environments.



(a) RDCs for `CartPole-v1` environment



(b) RDCs for `Hopper-v5` environment

Figure 2: Examples of RDCs. For both environments, the left plots correspond to observation delays and center plots to action delays. PPO's RDCs are highlighted in cyan since it is the algorithm that achieves the highest AUC scores. The right plots show return-delay surfaces from joint observation and action delays for PPO.

3.1 Delay-unaware algorithms

We explore the robustness of common RL algorithms to delayed observations and actions. These algorithms treat the delay-augmented environments as MDPs since they are unaware of the sensorimotor delays. Delay-unaware algorithms provide a useful performance baseline for the development of delay-aware approaches that alleviate performance degradation with increasing delays.

In our experiments we test the following algorithms, using their default implementations from the Stable-Baselines3 library [14]: Deep Q-Network (DQN) [15, 16], Proximal Policy Optimization (PPO) [17], Advantage Actor Critic (A2C) [18], Soft Actor Critic (SAC) [19], Deep Deterministic Policy Gradient (DDPG) [20], and Twin Delayed DDPG (TD3) [21]. DQN, PPO, and A2C are used in environments with discrete action spaces, whereas PPO, A2C, SAC, DDPG, and TD3 are used in environments with continuous action spaces. Additional details about the experiments, including algorithm hyperparameters, are provided in Appendix C.3.

3.2 Delay-aware algorithms

Delay-aware algorithms are those that attempt to compensate for the non-Markovianity of the environments. [12] categorizes delay-aware algorithms into five categories: state augmentations, recurrent policies, predictor-based strategies, domain-randomized training, and explicit constraint handling. The last two are most relevant in the contexts of risk-prevention and safe RL for robotics, and thus are not included in our experiments. Since PPO is the delay-unaware algorithm with the highest scores across all environments, we focus on delay-aware extensions of PPO.

Recurrent policy. A common approach to compensate for delays is to rely on memory-augmented architectures, typically implemented as policies with recurrent neural networks, namely Long Short-Term Memory (LSTM) architectures. The advantage of recurrent policies is that the hidden state of

Table 1: Delayed observation AUC scores for delay-unaware algorithms across multiple environments. The algorithm with the highest score for each environment is highlighted in **bold**.

Environment	DQN	PPO	A2C	SAC	DDPG	TD3
`Acrobot-v1`	228	**8078**	1060	–	–	–
`CartPole-v1`	396	**2046**	883	–	–	–
`Pendulum-v1`	–	**32687**	0	3647	3851	4110
`MountainCarContinuous-v0`	–	**1981**	700	4155	1070	5873
`InvertedPendulum-v5`	–	**3429**	1300	796	690	778
`InvertedDoublePendulum-v5`	–	6683	354	**7022**	5642	6471
`Pusher-v5`	–	**4519**	3834	2543	1004	1495
`Reacher-v5`	–	**1009**	585	618	832	431
`Ant-v5`	–	43431	**53769**	9968	0	383
`HalfCheetah-v5`	–	**24689**	1275	–	6541	–
`Hopper-v5`	–	**5499**	353	2901	1593	999
`Walker2d-v5`	–	**19883**	3236	6217	1215	3840
`Swimmer-v5`	–	977	413	**1140**	1004	920
`Humanoid-v5`	–	**6661**	632	1391	688	264

the LSTM integrates information over time, allowing the agent to infer the dynamics of its interactions with the environment. We use the default RecurrentPPO [6] implementation from Stable-Baselines3.

History stacking. Stacking the entire delay windows of states $s_{t-\tau_s}, \dots, s_t$ and actions $a_{t-\tau_a}, \dots, a_t$ recovers the Markov property of the environment [12]. However, the state information is not directly available to the agent, because at time t it has not yet observed states beyond $s_{t-\tau_s}$. On the other hand, the actions are accessible to the agent even if they have not been executed in the environment yet. In light of this, we propose a history stacking model where the agent's input is the concatenation of the delayed state and a history of the previous n selected actions:

$$\tilde{s}_t = s_{t-\tau_s} \oplus a_t \oplus a_{t-1} \oplus \ldots \oplus a_{t-n} \ . \tag{11}$$

The choice of n must be informed by the nature of the delays. In particular, for $\tau_a = 0$, setting $n = \tau_s$ could provide the agent with sufficient information to approximately predict the current state by exploiting the information from all the actions taken since the delayed state. For simplicity, we set $n = \max(\tau_s, \tau_a)$ and store the n actions in a FIFO buffer. We train the default PPO implementation from Stable-Baselines3 on the history stacking observations following Eq. 11.

Forward model. Finally, we also test forward models that explicitly predict the current state of the environment based on its delayed state and the selected actions. Similar approaches have gained considerable attention in recent years [7, 8, 9, 10], particularly those that rely on latent states for their predictions by means of world models [22, 23]. Predictor-based strategies are particularly promising if the agent is able to accurately anticipate the consequences of its actions, in which case the Markov property can be approximately recovered. Here, we consider a simple forward model f that predicts the environment's state a single step at a time using a window of three previous states and actions according to:

$$\hat{s}_t = f(s_{t-1}, a_{t-1}, s_{t-2}, a_{t-2}, s_{t-3}, a_{t-3}) \ . \tag{12}$$

For each environment we train a forward model that maps a window of three states and actions to the following state. The model is a multi-layer perceptron implemented in PyTorch [24] with three fully-connected hidden layers of size 256. It is trained from 1 million steps collected using random policy with a 80-20 split for training and validation. We train the forward models for a maximum of 1,000 epochs with early stopping using a learning rate with cosine scheduling between 10^{-4} and 10^{-5}. In Appendix B.1, we show that after training, the forward model can be used recursively to predict the environment's state up to the time $t - \tau_a$. In particular, if there is no action delay, then the forward model can predict the current state $\hat{s}_t$ regardless of the observation delay. In our experiments, we only use the forward model to compensate for observation delays. We train the default PPO implementation from Stable-Baselines3 on the predicted states $\hat{s}_t$.

Table 2: Delayed action AUC scores for delay-unaware algorithms across multiple environments. The algorithm with the highest score for each environment is highlighted in **bold**.

Environment	DQN	PPO	A2C	SAC	DDPG	TD3
`Acrobot-v1`	388	**7372**	867	–	–	–
`CartPole-v1`	129	**2646**	696	–	–	–
`Pendulum-v1`	–	**32686**	0	3578	3483	3670
`MountainCarContinuous-v0`	–	**20804**	210	611	1246	2138
`InvertedPendulum-v5`	–	**1763**	741	776	619	770
`InvertedDoublePendulum-v5`	–	6208	353	**6505**	5352	6061
`Pusher-v5`	–	**2750**	127	165	39	144
`Reacher-v5`	–	**689**	5	130	106	160
`Ant-v5`	–	**1923**	552	625	0	348
`HalfCheetah-v5`	–	**16172**	817	–	5168	–
`Hopper-v5`	–	**6527**	322	2245	1536	522
`Walker2d-v5`	–	**10679**	412	3354	1091	1550
`Swimmer-v5`	–	**1027**	97	408	477	987
`Humanoid-v5`	–	**3366**	547	1022	1667	262

4 Results

4.1 Delay-unaware algorithms' performance degrades under sensorimotor delays

DELAYBENCH allows us to characterize the robustness of RL algorithms under increasing observation and action delays by means of the RDC, a sigmoid function fitted to the evaluation returns after training. Fig. 2 shows the RDCs with observation and action delays of several common delay-unaware algorithms in two example environments with discrete (`CartPole-v1`) and continuous (`Hopper-v5`) control, respectively. In all cases, we find that performance degrades under sensorimotor delays. A comparison of the consequences of observation and action delays qualitative similarities between them. In fact, we do not identify any systematic differences between observation and action delays for delay-unaware algorithms. However, the characteristics of the degradation do vary between environments and algorithms. For example, PPO retains maximal performance in `CartPole-v1` but degrades almost immediately in `Hopper-v5`. Additionally, SAC appears to be less robust to delays than PPO. In Appendix D, we provide the RDCs of every algorithm, including the sigmoid parameters (slope and inflection point) as well as the quality of the fit.

By default, DELAYBENCH separates the evaluations of observation and action delays to isolate their effects on each environment and algorithm. Nonetheless, in Fig. 2 we also visualize the degradation of the return with concurrent observation and action delays. The return-delay surface is computed by fitting a two-dimensional sigmoid to the delays following a full grid of runs with the PPO algorithm on the `CartPole-v1` and `Hopper-v5` environments. In both cases, we find that the combination of observation and action delays has catastrophic consequences on the returns due to the aggregated effects of each.

4.2 PPO is more robust than other delay-unaware algorithms

We compare the robustness to delays of six popular delay-unaware RL algorithms: DQN, PPO, A2C, SAC, DDPG, and TD3. Their AUC scores with observation delays are shown in Table 1 and with action delays in Table 2. In both cases, we find that PPO consistently outperforms the other algorithms across most environments. In fact, PPO is not the highest scoring algorithm in only 4 out of 28 conditions. As revealed by the RDCs shown in Fig. 2, the high AUC scores of PPO are indicative of its robustness and not of a high zero-delay performance. By way of example, SAC achieves an average zero-delay return of 3055 while PPO only reaches an average zero-delay return of 1964, yet SAC has an observation delay AUC score of 2245 and PPO of 6527. To explain this difference we look at the RDC parameters of each algorithm. We find that that the inflection point of SAC is 0.738 and of PPO is 2.78. In other words, SAC's performance degrades considerably with a delay of a single step while PPO remains robust after delays of two steps. Previous work had

Table 3: Delayed observation and action AUC scores for the delay-unaware ("unaware") and different delay-aware versions ("recurrent": recurrent policy; "history": history stacking; "forward": forward model) of the PPO algorithm across multiple environments. The algorithm with the highest score for each environment and delay type is highlighted in **bold**.

	Observation delays				Action delays		
Environment	unaware	recurrent	history	forward	unaware	recurrent	history
`Acrobot-v1`	8078	337	11593	**82913**	7372	436	**11216**
`CartPole-v1`	2045	587	4945	**7052**	2646	397	**3572**
`Pendulum-v1`	**32687**	0	20954	10538	**15899**	0	8042
`Pusher-v5`	**4519**	3257	4160	4187	**2750**	1714	2507
`Reacher-v5`	1009	**1018**	878	1000	**689**	432	589
`Ant-v5`	**40753**	10789	19239	8506	2063	**3431**	511
`HalfCheetah-v5`	24689	18867	30084	**49199**	**16172**	9040	14326
`Hopper-v5`	5499	2132	7029	**16955**	6527	3788	**7514**
`Walker2d-v5`	19883	1215	**21754**	14532	**10679**	2193	9345
`Swimmer-v5`	977	527	1202	**1349**	**1027**	340	911
`Humanoid-v5`	6661	5843	5871	**7654**	**3366**	2909	3318

shown that PPO is more robust than other RL algorithms to perturbations, including changes to the environment [25]. Our results extend this finding to sensorimotor delays.

4.3 Delay robustness is only weakly related to zero-delay performance

To systematically study the relation between delay robustness and zero-delay performance, we aggregate the results from all the delay-unaware algorithms. Note from Eqs. 9 and 10 that if the degradation was the same for all experiments (i.e., same sigmoid slopes and inflection points), then the AUC score would be linearly correlated with the zero-delay return. We perform a linear regression between the empirical AUC scores and the average zero-delay returns to quantify the relation between delay robustness and zero-delay performance. We find a Pearson correlation coefficient $r = 0.11$ ($p > 0.05$), which indicates that the two quantities are not uncorrelated. We also evaluate the inflection point, which determines the mid-point of the RDC. As expected, we find a strong correlation between AUC score and inflection point ($r = 0.64,\ p < 0.001$) and a weak negative correlation between zero-delay performance and the inflection point ($r = -0.19,\ p > 0.001$). This negative correlation points to a possible trade-off between robustness to longer delays and zero-delay performance, as seen in Fig. 2 from the comparison between PPO and SAC in the `Hopper-v5` environment.

4.4 Delay-aware algorithms compensate for sensorimotor delays

Finally, we evaluate three delay-aware methods that extend the delay-unaware PPO algorithm with, respectively, recurrent policies, history stacking, and forward models. The resulting AUC scores for observation and action delays are shown in Table 3. With respect to the observation delays, the forward model that predicts the current state from the delayed state and the actions is the best overall algorithm, while the history stacking model also outperforms the delay-unaware PPO in a majority of environments. However, with action delays, the unaware model is the best algorithm. These results reveal that (i) performance degradation can be alleviated by correcting for the delays, but (ii) not every type of delay-aware method can compensate for every type of delay. In sum, these findings highlight how careful consideration is required in the design of delay-aware algorithms.

5 Discussion

Sensorimotor delays are ubiquitous in real-world systems but they have not been at the center of research and developments in RL. To facilitate the systematic study of sensorimotor delays in RL, we have proposed DELAYBENCH. DELAYBENCH makes use of general-purpose environments with

controlled observation and action delays as well as a standardized evaluation protocol with a scalar metric for robustness.

In our experiments with common RL algorithms, we have shown that observation and action delays can significantly impact learning, and that robustness is an independent characteristic from performance in the zero-delay condition. Delays play distinct roles in different environments and for different algorithms. Therefore, it is critical that researchers adequately characterize the degradation in performance of their algorithms in particular settings. Here, we observed that PPO is relatively robust to delays even if its performance at zero-delay is not the highest. This finding highlights the importance of making informed choices about the RL algorithm used in different context. We propose that PPO is a good candidate for real-world RL applications.

Beyond comparisons between common RL algorithms, we showed how DELAYBENCH can facilitate the development of new delay-aware algorithms that mitigate the effects of delays. In particular, we found that the use of a simple forward model that can predict future states is sufficient to minimize the degradation (see Fig. 22 in Appendix D). This positive result provides an encouraging perspective on the development of delay-aware models using DELAYBENCH as a systematic evaluation platform.

5.1 Limitations and future work

DELAYBENCH suffers from a number of limitations. Despite being motivated by the sensorimotor delays of real-world biological and robotic agents, DELAYBENCH only includes Gymnasium environments, spanning classic control and simple robotics and locomotion tasks from MuJoCo. This decision was purposefully made to keep the benchmark lightweight and general-purpose. Nonetheless, DELAYBENCH can be readily applied to environments that share the Gymnasium API. As a proof-of-concept, in Appendix B.2 we perform experiments with delay-augmented versions of the `PointMaze` environments from Gymnasium-robotics [26]. Future work should explore the use of delays in more sophisticated models of biological or robotic systems.

Another limitation is the simplification that DELAYBENCH only considers a single observation delay and a single action delay. In real biological and robotic systems, different sensors and actuators may have heterogeneous delays. For example, while the human vestibulo-ocular reflex couples sensation to action in under 10 ms [27], visual reaction times (from visual stimulus onset to a subject's button press) are of the order of a quarter second [28]. Permitting heterogeneous delays for different sensory and action modalities is an interesting direction for future work.

A further limitation of our current implementation is the initialization of the action buffer with random actions at the beginning of an episode. In tasks such as pole balancing, an initial series of random actions can be very harmful. In such settings, it would be interesting to offer the option of initializing the action buffer with a constant "default" action such as not moving at all.

6 Conclusion

We have presented DELAYBENCH, a benchmark for evaluating RL algorithms under observation and action delays as they commonly arise in real robots or biological systems. Robustness to delays is quantified by evaluating algorithms' returns for different delays and summarizing them into a single metric, the area under the return-delay curve. DELAYBENCH extends Gymnasium environments with observation and action delays while retaining the original API, allowing users to train and test models with minimal efforts. We propose DELAYBENCH as a standard evaluation benchmark for general-purpose RL and particularly for delay-aware algorithms. We hope that this will ultimately contribute to a better understanding of how robots and biological systems can mitigate the effects of omnipresent sensorimotor delays.

Acknowledgments and Disclosure of Funding

This work was supported by the Deutsche Forschungsgemeinschaft (German Research Foundation, DFG) under Germany's Excellence Strategy (EXC 3066/1 "The Adaptive Mind", Project No. 533717223). J.T. was supported by the Johanna Quandt foundation. Experiments were performed on the computational cluster Katana supported by Research Technology Services at UNSW Sydney.

References

[1] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[2] Robert Whelan. Effective analysis of reaction time data. *The psychological record*, 58(3):475–482, 2008.

[3] CD Manning, SA Tolhurst, and Parveen Bawa. Proprioceptive reaction times and long-latency reflexes in humans. *Experimental brain research*, 221(2):155–166, 2012.

[4] Olivier Codol, Mehrdad Kashefi, Christopher J Forgaard, Joseph M Galea, J Andrew Pruszynski, and Paul L Gribble. Sensorimotor feedback loops are selectively sensitive to reward. *Elife*, 12:e81325, 2023.

[5] Matthew J Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *AAAI fall symposia*, volume 45, page 141, 2015.

[6] Marco Pleines, Matthias Pallasch, Frank Zimmer, and Mike Preuss. Generalization, mayhems and limits in recurrent proximal policy optimization. *arXiv preprint arXiv:2205.11104*, 2022.

[7] Armin Karamzade, Kyungmin Kim, Montek Kalsi, and Roy Fox. Reinforcement learning from delayed observations via world models. *arXiv preprint arXiv:2403.12309*, 2024.

[8] Armin Karamzade, Kyungmin Kim, JB Lanier, Davide Corsi, and Roy Fox. Model-based reinforcement learning under random observation delays. *arXiv preprint arXiv:2509.20869*, 2025.

[9] Qingyuan Wu, Yuhui Wang, Simon Sinong Zhan, Yixuan Wang, Chung-Wei Lin, Chen Lv, Qi Zhu, Jürgen Schmidhuber, and Chao Huang. Directly forecasting belief for reinforcement learning with delays. *arXiv preprint arXiv:2505.00546*, 2025.

[10] Baiming Chen, Mengdi Xu, Liang Li, and Ding Zhao. Delay-aware model-based reinforcement learning for continuous control. *Neurocomputing*, 450:119–128, 2021.

[11] Bo Xia, Yilun Kong, Yongzhe Chang, Bo Yuan, Zhiheng Li, Xueqian Wang, and Bin Liang. Delay-aware reinforcement learning with encoder-enhanced state representations. *IEEE Transactions on Artificial Intelligence*, 2026.

[12] Armando Alves Neto. Reinforcement learning for control systems with time delays: A comprehensive survey. *arXiv preprint arXiv:2602.00399*, 2026.

[13] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[14] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of machine learning research*, 22(268):1–8, 2021.

[15] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[16] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[18] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.

[19] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.

[20] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. arxiv 2015. *arXiv preprint arXiv:1509.02971*, 2015.

[21] Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018.

[22] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.

[23] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.

[24] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[25] Thomas Nakken Larsen, Halvor Ødegård Teigen, Torkel Laache, Damiano Varagnolo, and Adil Rasheed. Comparing deep reinforcement learning algorithms' ability to safely navigate challenging waters. *Frontiers in Robotics and AI*, 8:738113, 2021.

[26] Rodrigo de Lazcano, Kallinteris Andreas, Jun Jet Tai, Seungjae Ryan Lee, and Jordan Terry. Gymnasium robotics, 2024.

[27] ST Aw, GM Halmagyi, T Haslwanter, IS Curthoys, RA Yavor, and MJ Todd. Three-dimensional vector analysis of the human vestibuloocular reflex in response to high-acceleration head rotations. ii. responses in subjects with unilateral vestibular loss and selective semicircular canal occlusion. *Journal of neurophysiology*, 76(6):4021–4030, 1996.

[28] Aditya Jain, Ramta Bansal, Avnish Kumar, and KD Singh. A comparative study of visual and auditory reaction times on the basis of gender and physical activity levels of medical first year students. *International journal of applied and basic medical research*, 5(2):124–127, 2015.

[29] Frédéric Crevecoeur and Michel Gevers. Filtering compensation for delays and prediction errors during sensorimotor control. *Neural computation*, 31(4):738–764, 2019.

[30] Daniel McNamee and Daniel M Wolpert. Internal models in biological control. *Annual review of control, robotics, and autonomous systems*, 2(1):339–364, 2019.

[31] Brandon G Rasman, Jean-Sébastien Blouin, Amin M Nasrabadi, Remco van Woerkom, Maarten A Frens, and Patrick A Forbes. Learning to stand with sensorimotor delays generalizes across directions and from hand to leg effectors. *Communications Biology*, 7(1):384, 2024.

[32] Jing Du, William Vann, Tianyu Zhou, Yang Ye, and Qi Zhu. Sensory manipulation as a countermeasure to robot teleoperation delays: system and evidence. *Scientific Reports*, 14(1):4333, 2024.

[33] Sibo Wang-Chen, Victor Alfred Stimpfling, Thomas Ka Chung Lam, Pembe Gizem Özdil, Louise Genoud, Femke Hurtak, and Pavan Ramdya. Neuromechfly v2: simulating embodied sensorimotor control in adult drosophila. *Nature Methods*, 21(12):2353–2362, 2024.

[34] Diego Aldarondo, Josh Merel, Jesse D Marshall, Leonard Hasenclever, Ugne Klibaite, Amanda Gellis, Yuval Tassa, Greg Wayne, Matthew Botvinick, and Bence P Ölveczky. A virtual rodent predicts the structure of neural activity across behaviours. *Nature*, 632(8025):594–602, 2024.

[35] Dominik Mattern, Pierre Schumacher, Francisco M López, Marcel C Raabe, Markus R Ernst, Arthur Aubret, and Jochen Triesch. Mimo: A multimodal infant model for studying cognitive development. *IEEE Transactions on Cognitive and Developmental Systems*, 16(4):1291–1301, 2024.

[36] Carmelo Sferrazza, Dun-Ming Huang, Xingyu Lin, Youngwoon Lee, and Pieter Abbeel. Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation. *arXiv preprint arXiv:2403.10506*, 2024.

[37] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998.

[38] Gabriel Dulac-Arnold, Nir Levine, Daniel J. Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. An empirical investigation of the challenges of real-world reinforcement learning. 2020.

[39] Gabriel Dulac-Arnold, Daniel Mankowitz, and Todd Hester. Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901*, 2019.

[40] Gabriel Dulac-Arnold, Nir Levine, Daniel J Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. An empirical investigation of the challenges of real-world reinforcement learning. *arXiv preprint arXiv:2003.11881*, 2020.

[41] Beining Han, Zhizhou Ren, Zuofan Wu, Yuan Zhou, and Jian Peng. Off-policy reinforcement learning with delayed rewards. In *International conference on machine learning*, pages 8280–8303. PMLR, 2022.

[42] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013.

[43] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012.

[44] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

[45] Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. Popgym: Benchmarking partially observable reinforcement learning. *arXiv preprint arXiv:2303.01859*, 2023.

[46] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning, 2020.

[47] Alex Ray, Joshua Achiam, and Dario Amodei. Benchmarking safe exploration in deep reinforcement learning. *arXiv preprint arXiv:1910.01708*, 7(1):2, 2019.

[48] Matteo Bettini, Amanda Prorok, and Vincent Moens. Benchmarl: Benchmarking multi-agent reinforcement learning. *Journal of Machine Learning Research*, 25(217):1–10, 2024.

A Related works

Sensorimotor delays in real-world agents. Sensorimotor delays are unavoidable for biological and robotic agents. By way of example, biological delays include sensory transduction, signal conduction at finite velocities in neural fibers, and muscle response latencies to motor commands, all of which amount to reaction times of tens or hundreds of milliseconds [2, 3]. Humans and non-human animals are thought to compensate for such delays through the use of internal models [4, 29, 30, 31]. Robotic systems face analogous delays. To alleviate the harmful effects of such delays, robotics engineers must choose between two approaches: using faster sensing and actuation components and interfaces (hardware solution) or using smarter learning algorithms (software solution). The former solution is usually expensive, which is why there is increasing interest in the second one, e.g. [32]. DELAYBENCH can be a suitable testing platform for computational models of real-world delay compensation (see [30] for a review). To do so, DELAYBENCH can be combined with other simulators of biological [33, 34, 35] or robotic [26, 36] systems that make use of the Gymnasium API.

RL with observation and action delays. Delays are also a challenge in RL, although they have not been traditionally considered as part of the systematic evaluations of new algorithms. Observation or action delays turn Markovian environments into POMDPs with temporal misalignment. Early approaches to such sensorimotor delays included augmented states with observation histories and recurrent policies [37, 5, 6]. More recently, following the success of world models in RL [22, 23], there has been a surge in model-based predictive approaches, where sensorimotor delays can be compensated in latent states, e.g. [7, 8, 9, 10, 11]. For a recent review of RL with observation and action delays, see [12]. Despite these recent efforts to develop delay-aware RL algorithms, most works rely on their own delay-augmented environments and measure the success of their algorithms following custom evaluations. There is currently no standardized protocol or benchmark for RL algorithms under sensorimotor delays. The closest tool is the Real-World RL (RWRL) suite [38], which includes sensorimotor delays as well as other perturbations inspired in real-world systems [39]. While the RWRL suite has been used to test models under sensorimotor delays [40], it does not provide a standardized evaluation protocol that can be systematically applied to compare existing and new algorithms. DELAYBENCH intends to fill this gap. Our experiments show that common algorithms are sensitive to moderate delays and that simple delay-aware extensions can be sufficient to alleviate this degradation in performance.

Delayed rewards and credit assignment. Other than observations and actions, temporal delays in RL can also influence the rewards received by the agent from the environment. This poses a separate problem for RL algorithms, typically described as credit assignment [1]. Delayed rewards affect how agents attribute positive or negative values to earlier decisions. In that regard, artificially delayed rewards are not different from naturally delayed rewards, such as a sparse success or failure signal in a game of chess. While the problem of delayed rewards is an active field of study in RL, e.g. [41], we decide to keep it separate from our focus on delayed observations and actions in DELAYBENCH.

RL benchmarks. Like other subfields of machine learning, progress in RL has been largely driven by a small set of widely used benchmark suites. Among these are the Arcade Learning Environment (Atari) [42], MuJoCo continuous control environments [43], the DeepMind control suite [44], and recently Gymnasium [13]. These tools are ideally suited for testing general-purpose RL algorithms. In parallel, additional efforts have been made in recent years to develop benchmarks with very specific focus areas, including partial observability [45], offline RL [46], safe RL [47], or multi-agent RL [48]. DELAYBENCH extends this list with a focus on sensorimotor delays.

B Supplementary results

B.1 Forward model predictions

The PPO models trained on the predictions of the forward models achieved, overall, the highest AUC scores under delayed observations. The RDC plots shown in Fig. 22 reveal that for most environments there is only marginal performance degradation. To understand the origin of this result, we plot the prediction errors of the forward models for all environments in Fig. A, aggregated over 100 random

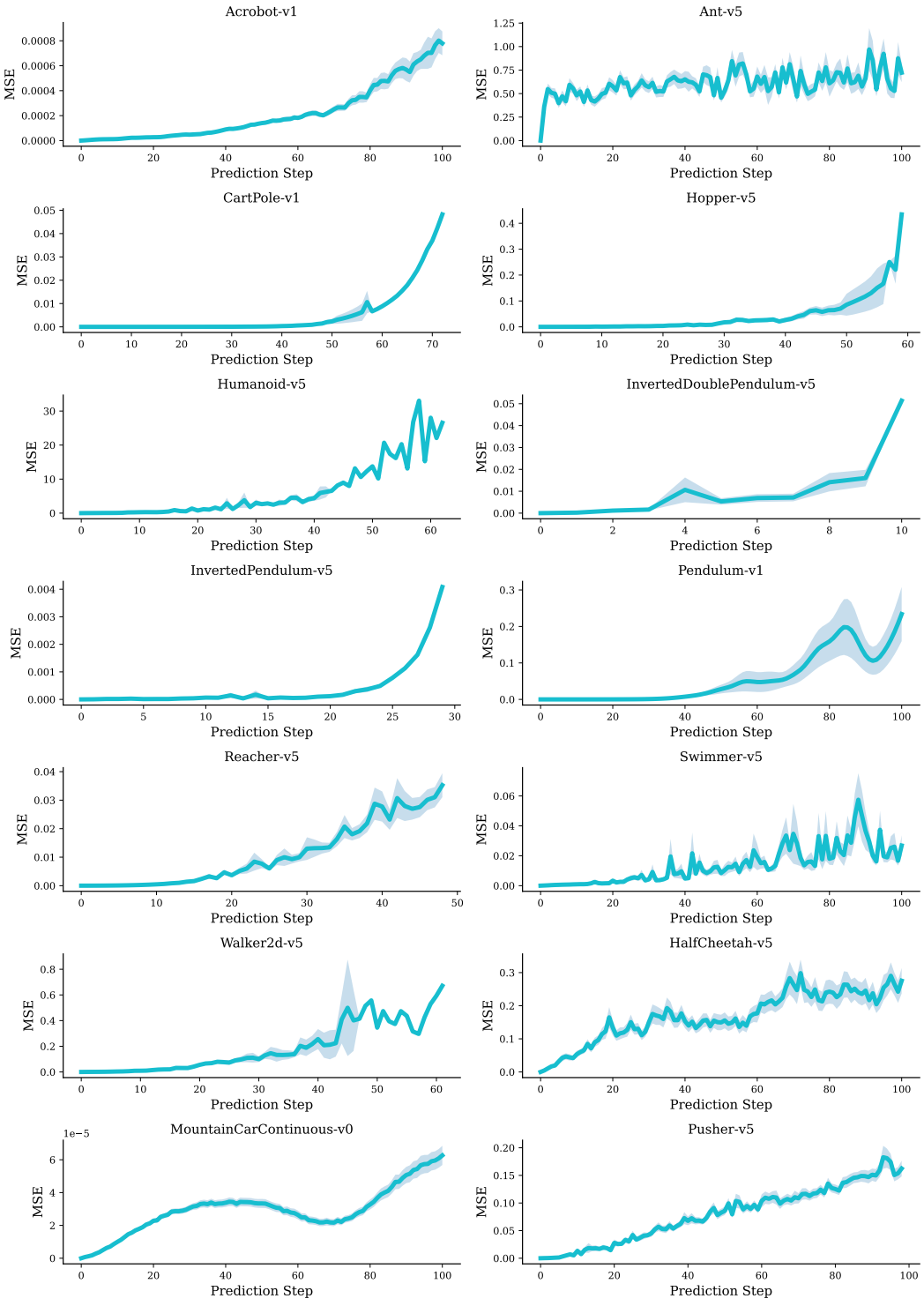
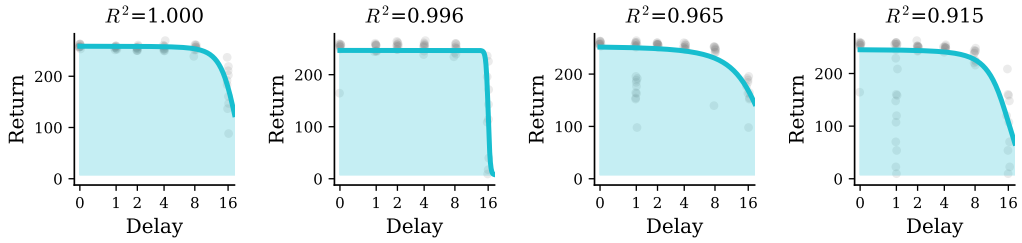


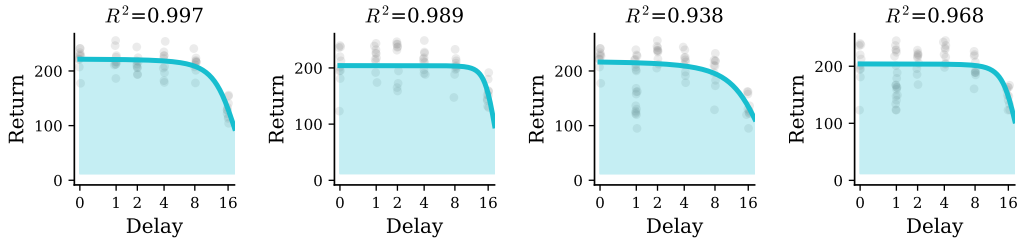
Figure 3: Forward model prediction errors over 100 steps aggregated over 100 random seeds. The shaded area indicates the standard error.

seeds, and for a maximum of 100 steps. In each run, the agent performed actions sampled uniformly from its action space, and the forward model relied exclusively on its predictions. In most cases, the prediction error remains close to zero for durations beyond 16 steps, the largest observation delay in our experiments. In sum, while the prediction errors accumulate over time and can reach large



(a) Unaware, observations (b) History, observations (c) Unaware, actions (d) History, actions

Figure 4: RDCs for the `PointMaze_Open-v3` environment



(a) Unaware, observations (b) History, observations (c) Unaware, actions (d) History, actions

Figure 5: RDCs for the `PointMaze_UMaze-v3` environment

Table 4: Delayed observation and action AUC scores for the `PointMaze` environments. The algorithm with the highest score for each environment and delay type is highlighted in **bold**.

	Observation delays		Action delays	
Environment	unaware	history	unaware	history
`PointMaze_Open-v3`	**4501**	3823	**4854**	3591
`PointMaze_UMaze-v3`	**3550**	3428	3469	**3591**

values for very long delays, the forward model remains accurate in the moderate delay regime of DELAYBENCH.

B.2 `PointMaze` environments

To showcase the possibility of using DELAYBENCH with other RL environments, we train the delay-unaware PPO and the history stacking PPO algorithms on the `PointMaze_Open-v3` and `PointMaze_UMaze-v3` from the Gymnasium-robotics library [26]. Unlike the Gymnasium environments, these `PointMaze` environments have dictionary observation spaces which include information about the state of the environment and about desired goals. This format is common to all environments in Gymnasium-robotics. DELAYBENCH can correctly apply delays for these environments, retaining the same API.

The AUC scores of the `PointMaze` environments with delayed observations and actions are shown in Table 4, and their RDCs are shown in Fig. 4. In all cases, the algorithms are very robust to delays.

C Experiment details

C.1 Compute resources

We trained every combination of environment, algorithm, and delay using 20 random seeds. All observation spaces were z-scored to zero mean and unit variance using 1 million observations collected

from a random policy. The training time was fixed as a maximum number of RL steps, which varied for each environment (see below) and was determined from pilot runs. In total, we performed 16,800 runs for the delay-unaware algorithms and 11,760 runs for the delay-aware algorithms, amounting to 28,560 total runs for the results reported in the main text, with additional runs for the results reported in the Appendix. All experiments were run on CPU nodes of a university high-performance computing cluster, with each training run being allocated 4 CPU cores and 12 GB of memory.

C.2 Environments

The full list of environments with details about their training times as well as observation and action spaces is provided below.

CartPole-v1 The algorithms are trained for a total of 50000 environment steps. Each episode has a maximum duration of 500. The action space is discrete with dimension 1, and the observation space has dimension 4.

Acrobot-v1. The algorithms are trained for a total of 150000 environment steps. Each episode has a maximum duration of 500. The action space is discrete with dimension 1, and the observation space has dimension 6.

Pendulum-v1. The algorithms are trained for a total of 150000 environment steps. Each episode has a maximum duration of 200. The action space is continuous with dimension 1, and the observation space has dimension 3.

MountainCarContinuous-v0. The algorithms are trained for a total of 150000 environment steps. Each episode has a maximum duration of 999. The action space is continuous with dimension 1, and the observation space has dimension 2.

InvertedPendulum-v5. The algorithms are trained for a total of 150000 environment steps. Each episode has a maximum duration of 1000. The action space is continuous with dimension 1, and the observation space has dimension 4.

InvertedDoublePendulum-v5. The algorithms are trained for a total of 200000 environment steps. Each episode has a maximum duration of 1000. The action space is continuous with dimension 1, and the observation space has dimension 9.

Ant-v5. The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 1000. The action space is continuous with dimension 8, and the observation space has dimension 105.

HalfCheetah-v5. The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 1000. The action space is continuous with dimension 6, and the observation space has dimension 17.

Hopper-v5. The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 1000. The action space is continuous with dimension 3, and the observation space has dimension 11.

Walker2d-v5. The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 1000. The action space is continuous with dimension 6, and the observation space has dimension 17.

Swimmer-v5. The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 1000. The action space is continuous with dimension 2, and the observation space has dimension 8.

Humanoid-v5 The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 1000. The action space is continuous with dimension 17, and the observation space has dimension 348.

Pusher-v5. The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 100. The action space is continuous with dimension 7, and the observation space has dimension 23.

Reacher-v5. The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 50. The action space is continuous with dimension 2, and the observation space has dimension 10.

PointMaze_Open-v3. The algorithms are trained for a total of 200000 environment steps. Each episode has a maximum duration of 100. The action space is continuous with dimension 2, and the observation space is a dictionary that contains the observation with dimension 4, the desired goal with dimension 2 and the achieved goal with dimension 2.

PointMaze_UMaze-v3:. The algorithms are trained for a total of 300000 environment steps. Each episode has a maximum duration of 100. The action space is continuous with dimension 2, and the observation space is a dictionary that contains the observation with dimension 4, the desired goal with dimension 2 and the achieved goal with dimension 2.

C.3 RL algorithms

All the RL algorithms in the experiments were used with their default implementations from Stable-Baselines3, including the default hyperparameters. The values are detailed below.

DQN. learning rate=0.0001, buffer size=1000000, learning starts=100, batch size=32, tau=1.0, gamma=0.99, train freq=4, gradient steps=1, n steps=1, target update interval=10000, exploration fraction=0.1, exploration initial eps=1.0, exploration final eps=0.05, max grad norm=10

PPO. learning rate=0.0003, n steps=2048, batch size=64, n epochs=10, gamma=0.99, gae lambda=0.95, clip range=0.2, normalize advantage=True, ent coef=0.0, vf coef=0.5, max grad norm=0.5

A2C. learning rate=0.0007, n steps=5, gamma=0.99, gae lambda=1.0, ent coef=0.0, vf coef=0.5, max grad norm=0.5, rms prop eps=1e-05

SAC. learning rate=0.0003, buffer size=1000000, learning starts=100, batch size=256, tau=0.005, gamma=0.99, train freq=1, gradient steps=1

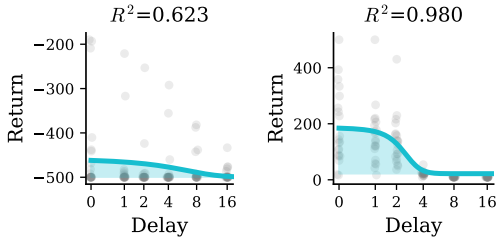
DDPG. learning rate=0.001, buffer size=1000000, learning starts=100, batch size=256, tau=0.005, gamma=0.99, train freq=1, gradient steps=1, n steps=1

TD3. learning rate=0.001, buffer size=1000000, learning starts=100, batch size=256, tau=0.005, gamma=0.99, train freq=1, gradient steps=1, n steps=1, policy delay=2, target policy noise=0.2, target noise clip=0.5

D Return-delay curves

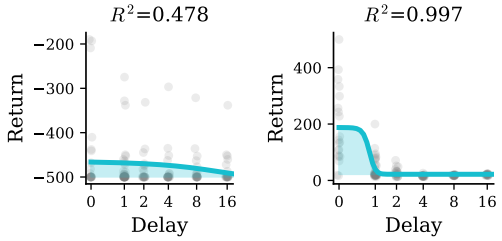
We show the RDCs for all pairs of environments and algorithms trained. Each plot includes the individual evaluation returns from single runs as gray scattered dots and the sigmoid fit in cyan. The shaded area under the sigmoid corresponds to the AUC score for that experiment. Each plot's title indicates the R^2 score of the sigmoid fit to the empirical returns.

DQN:



(a) `Acrobot-v1` (b) `CartPole-v1`

Figure 6: Observation-delayed RDCs for delay-unaware DQN



(a) `Acrobot-v1` (b) `CartPole-v1`

Figure 7: Action-delayed RDCs for delay-unaware DQN

PPO:

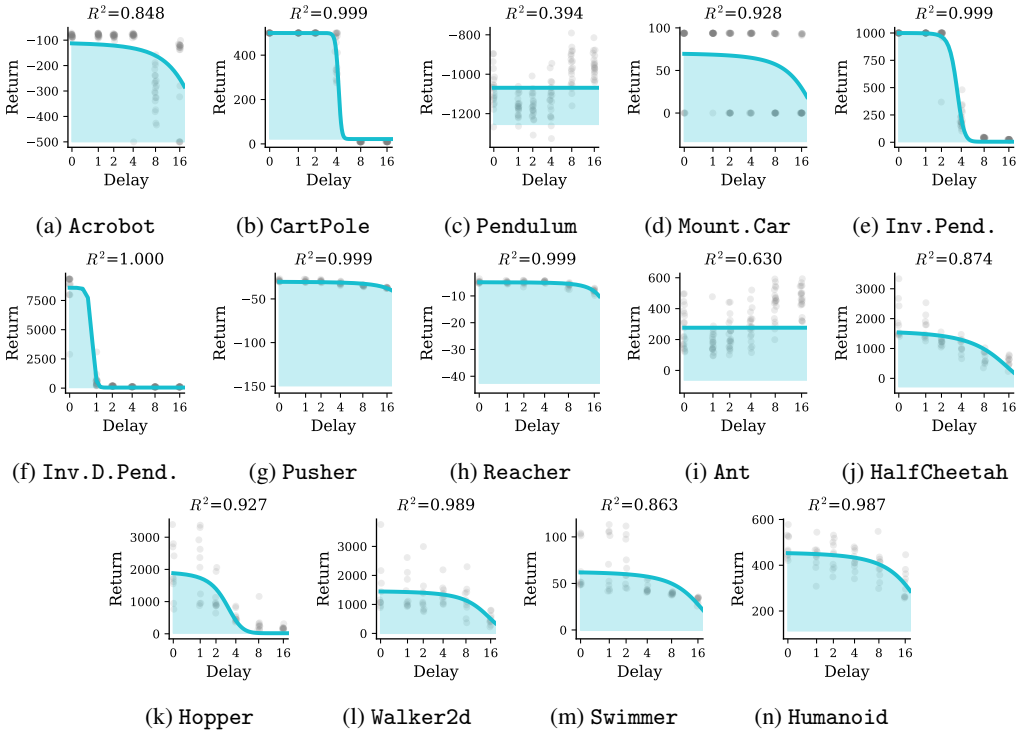


Figure 8: Observation-delayed RDCs for delay-unaware PPO

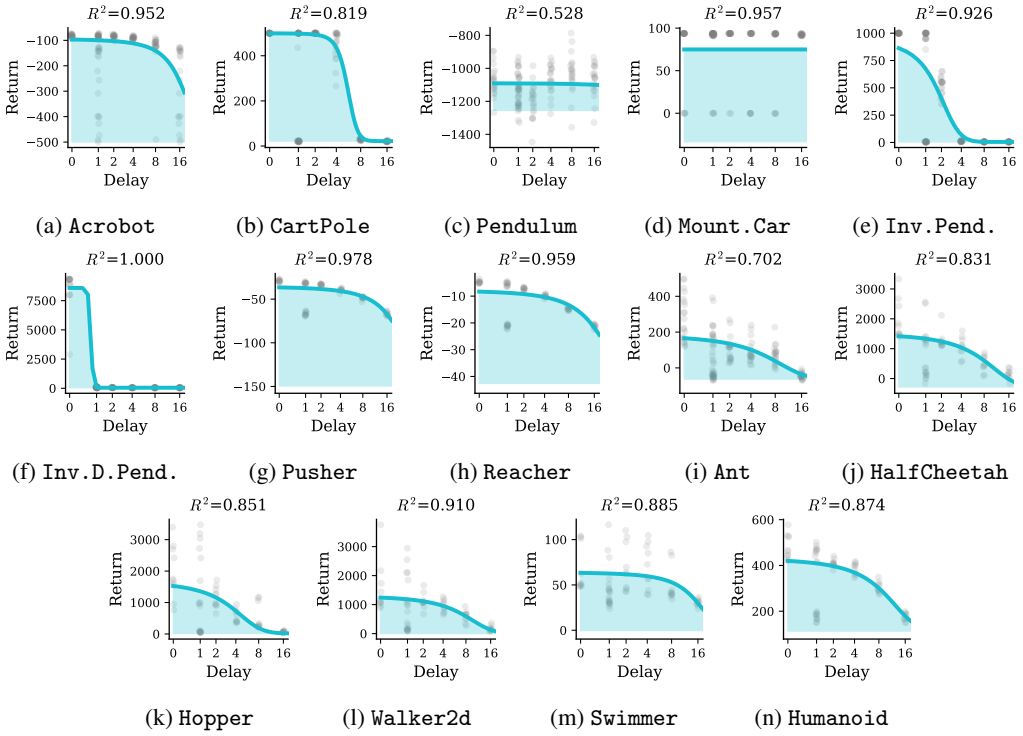


Figure 9: Action-delayed RDCs for delay-unaware PPO

A2C:

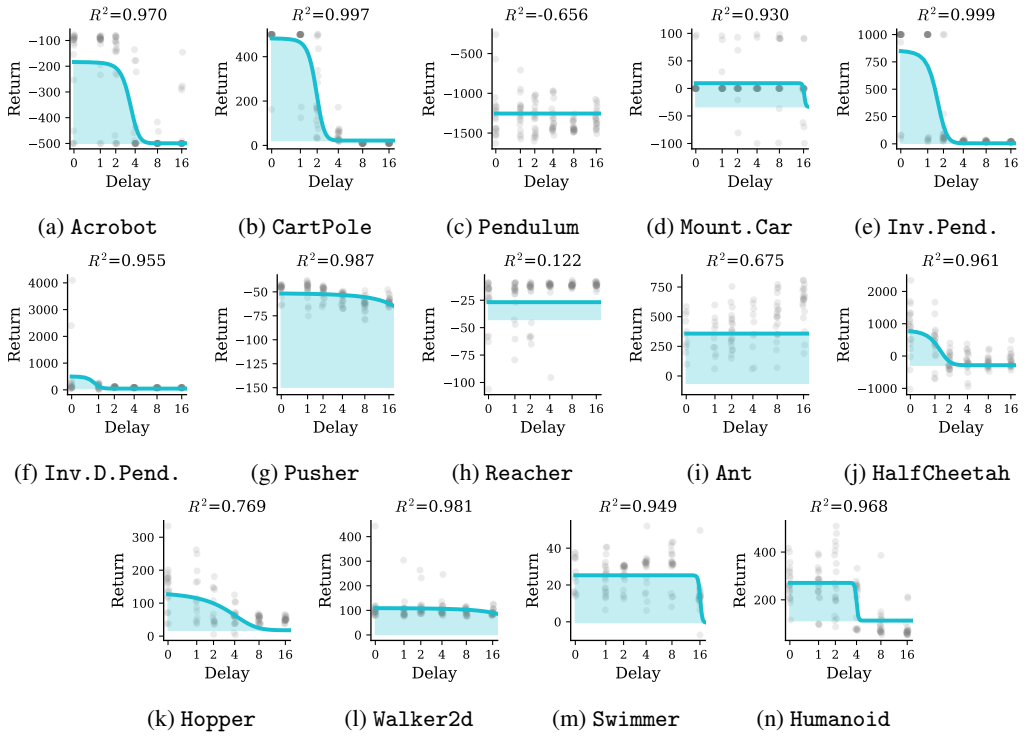


Figure 10: Observation-delayed RDCs for delay-unaware A2C

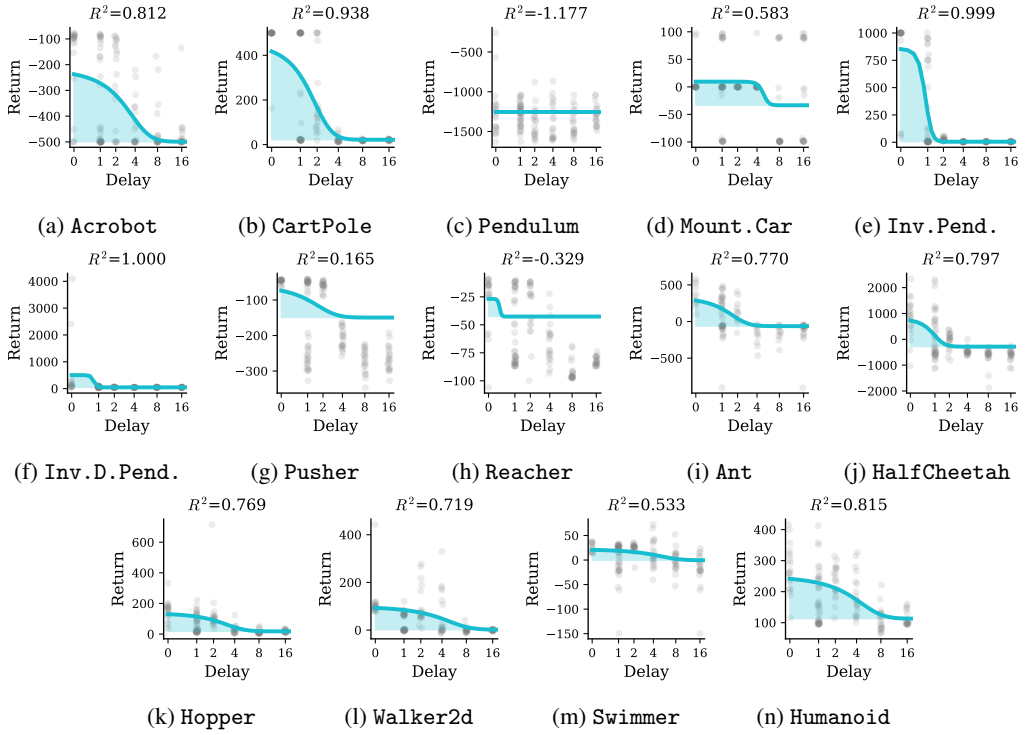


Figure 11: Action-delayed RDCs for delay-unaware A2C

SAC:

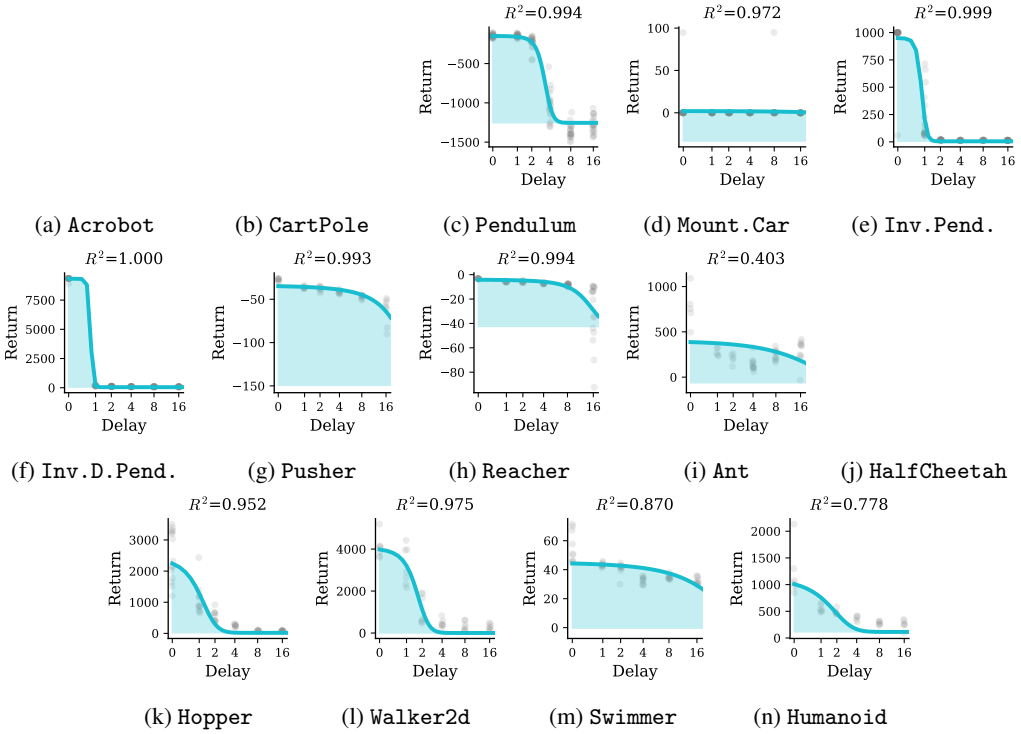


Figure 12: Observation-delayed RDCs for delay-unaware SAC

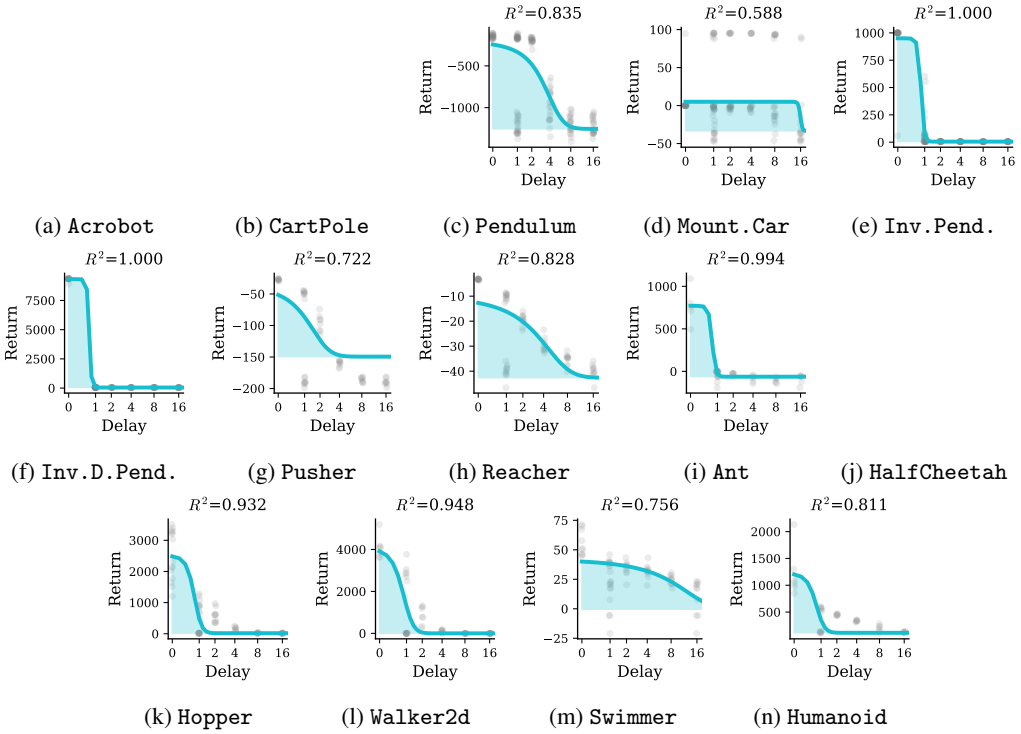


Figure 13: Action-delayed RDCs for delay-unaware SAC

DDPG:

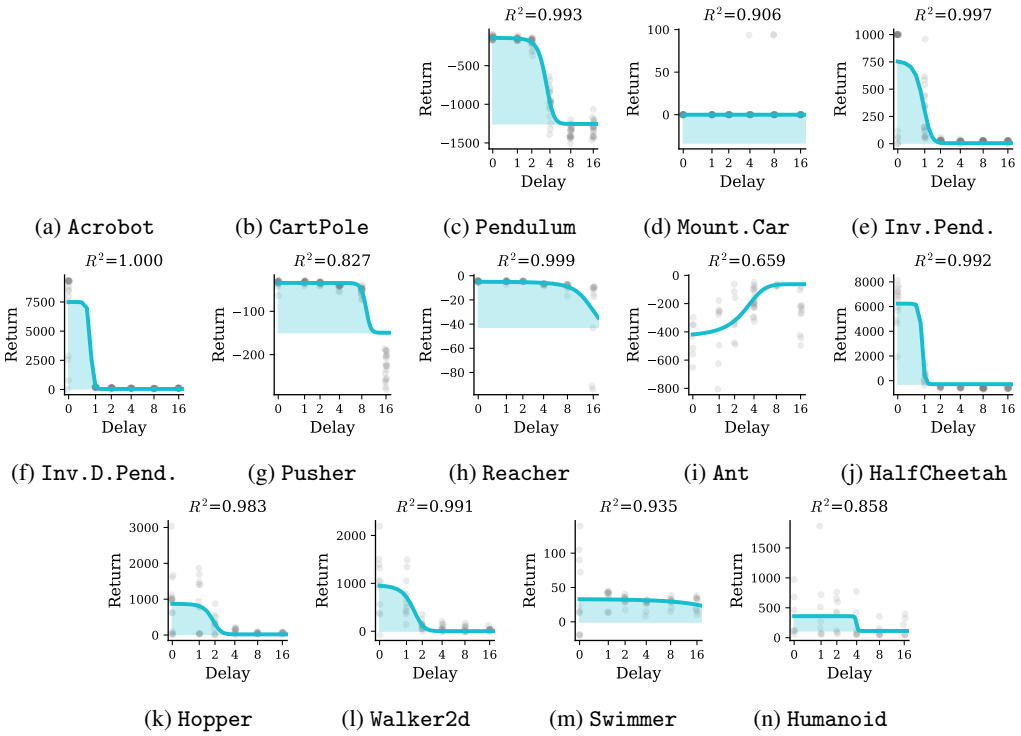


Figure 14: Observation-delayed RDCs for delay-unaware DDPG

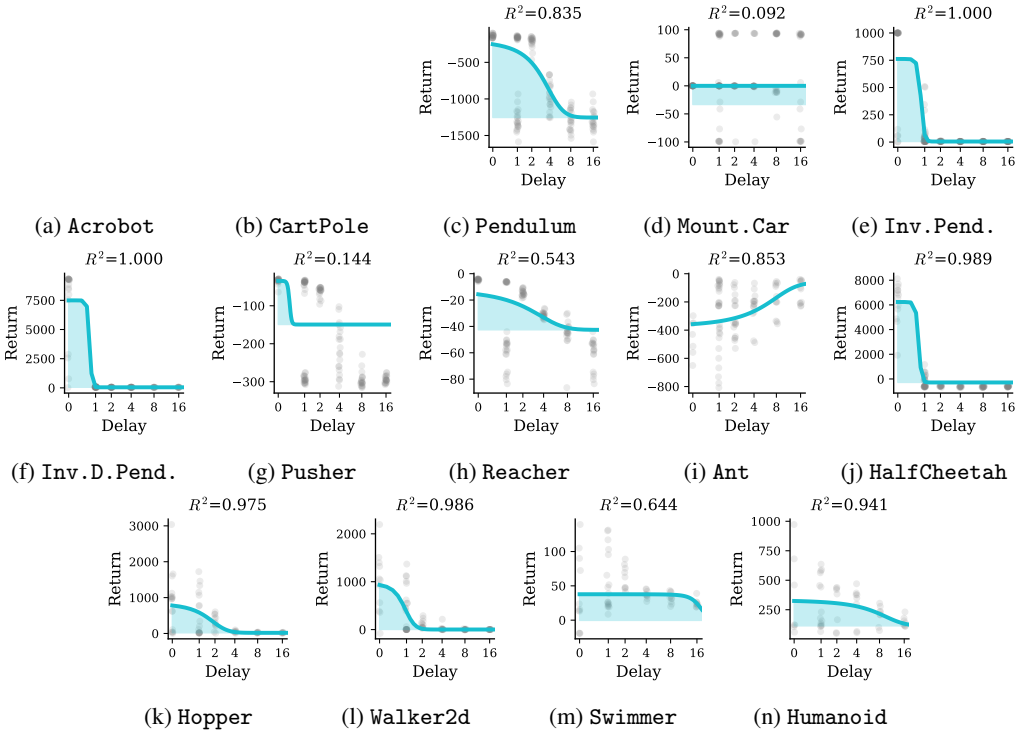


Figure 15: Action-delayed RDCs for delay-unaware DDPG

TD3:

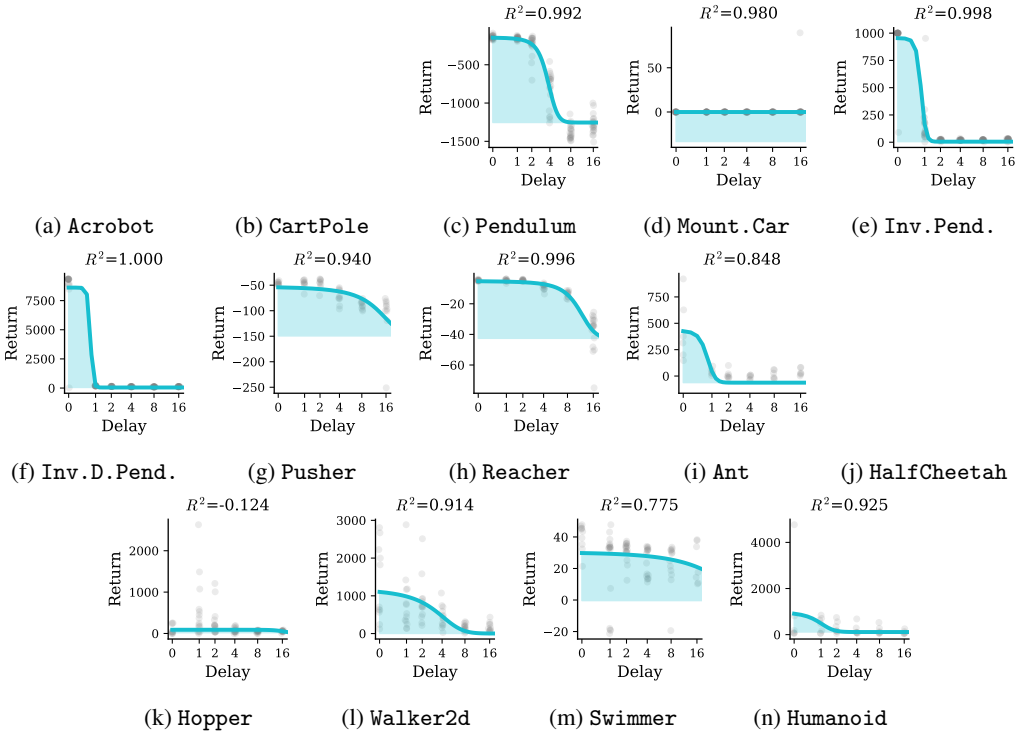


Figure 16: Observation-delayed RDCs for delay-unaware TD3

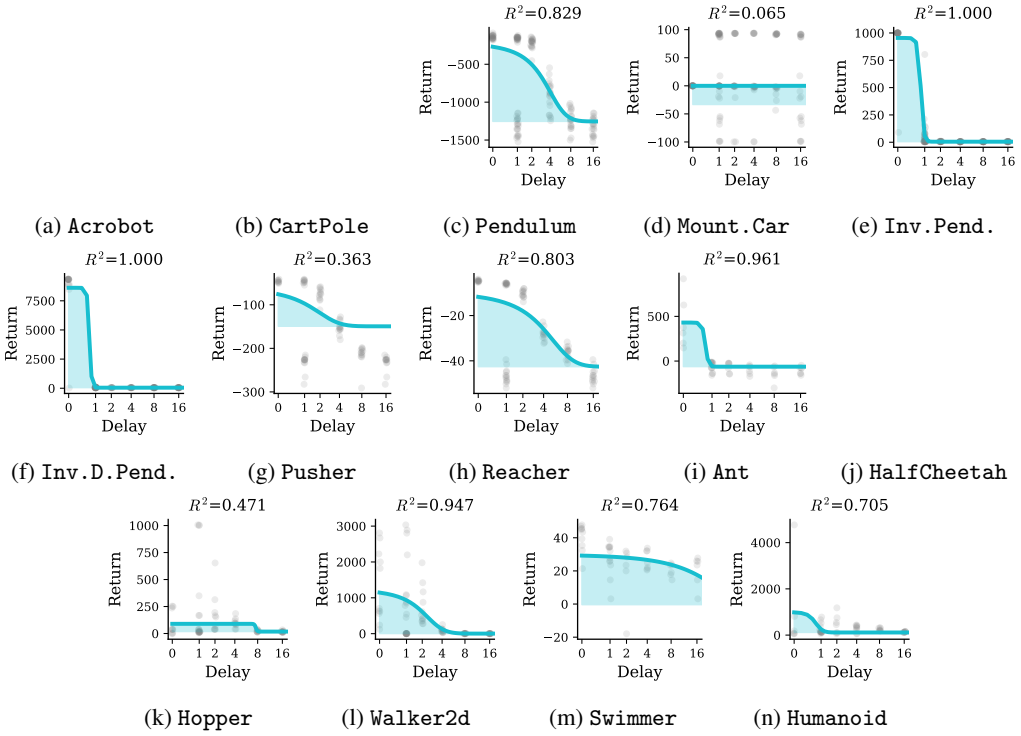
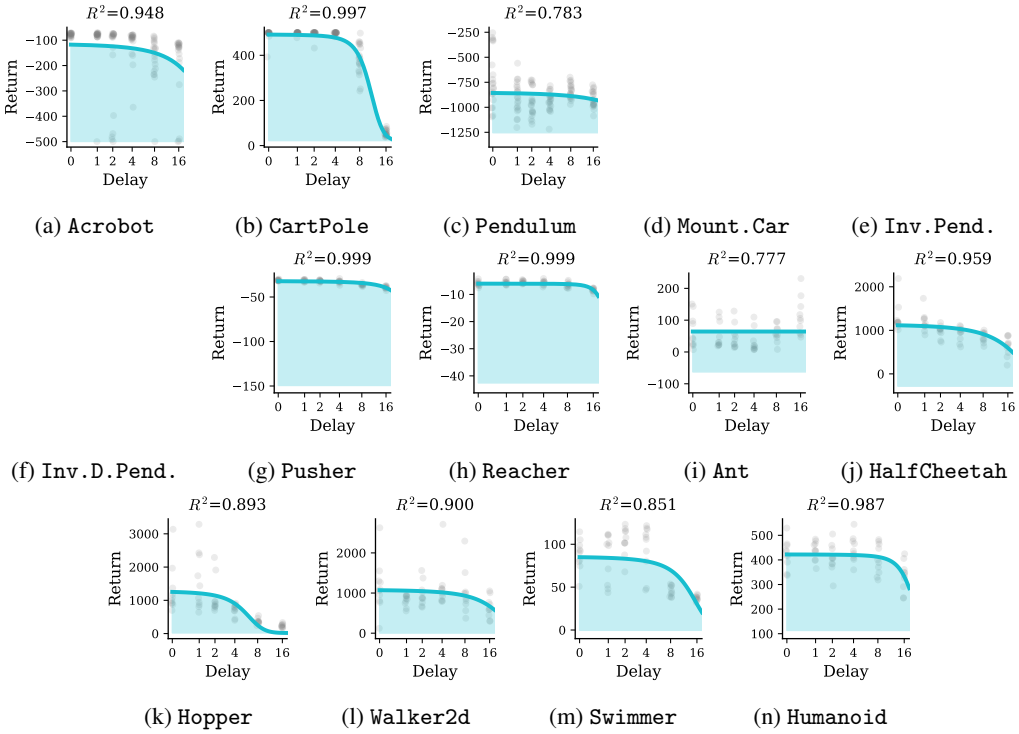


Figure 17: Action-delayed RDCs for delay-unaware TD3

History stacking PPO:

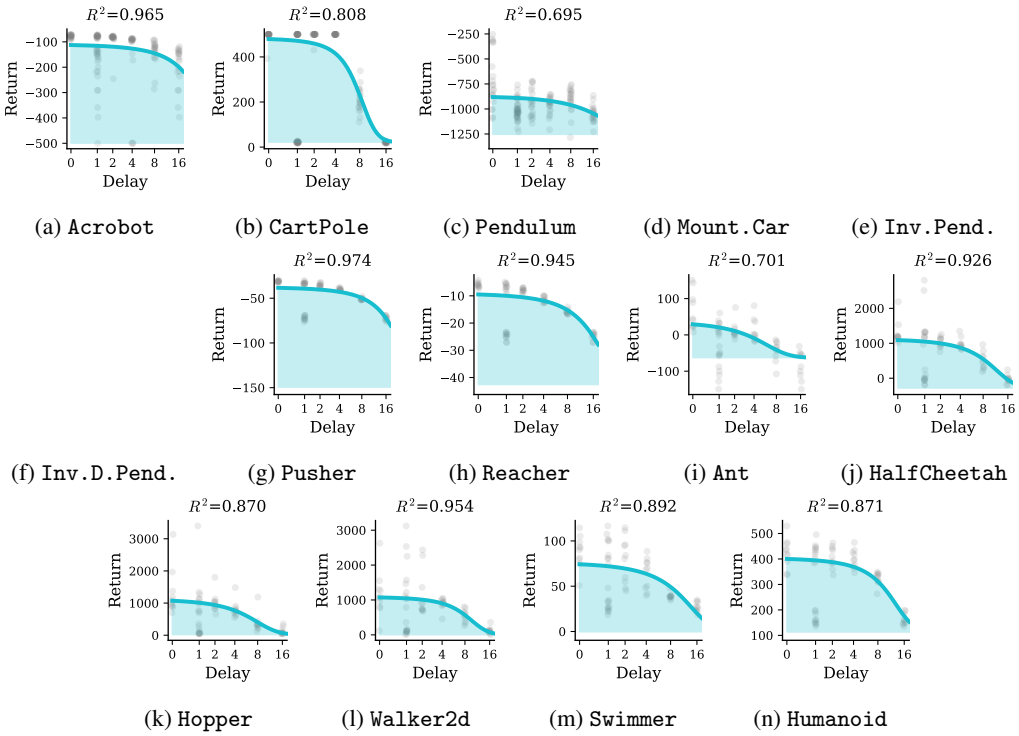


(a) `Acrobot` (b) `CartPole` (c) `Pendulum` (d) `Mount.Car` (e) `Inv.Pend.`

(f) `Inv.D.Pend.` (g) `Pusher` (h) `Reacher` (i) `Ant` (j) `HalfCheetah`

(k) `Hopper` (l) `Walker2d` (m) `Swimmer` (n) `Humanoid`

Figure 18: Observation-delayed RDCs for history stacking PPO



(a) `Acrobot` (b) `CartPole` (c) `Pendulum` (d) `Mount.Car` (e) `Inv.Pend.`

(f) `Inv.D.Pend.` (g) `Pusher` (h) `Reacher` (i) `Ant` (j) `HalfCheetah`

(k) `Hopper` (l) `Walker2d` (m) `Swimmer` (n) `Humanoid`

Figure 19: Action-delayed RDCs for history stacking PPO

RecurrentPPO:

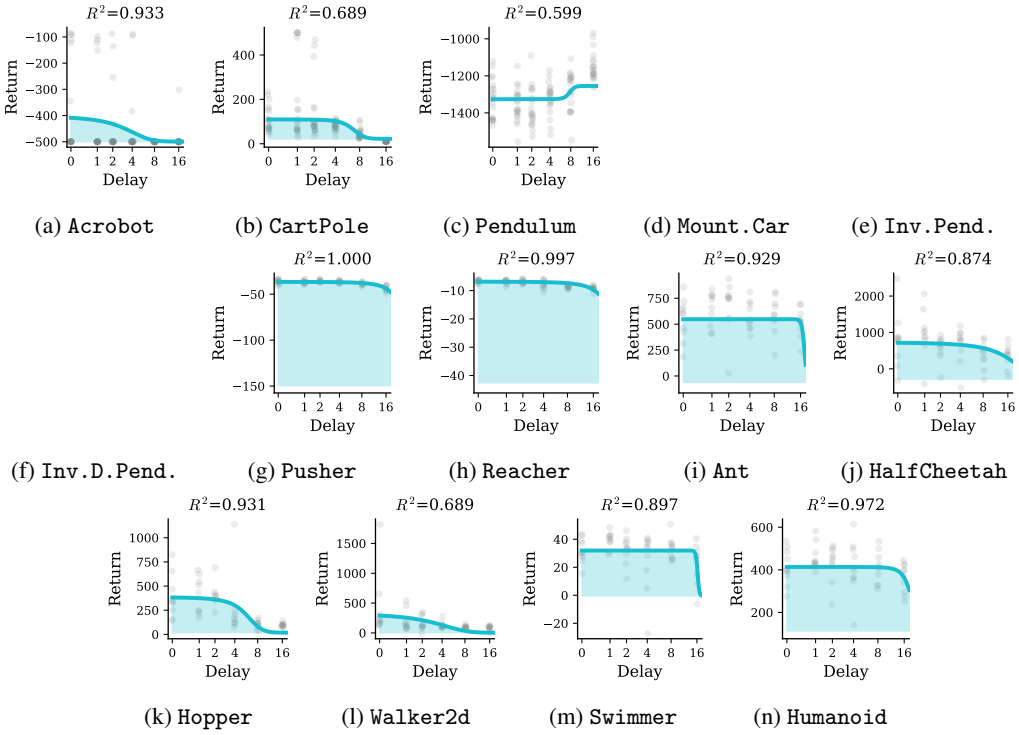


Figure 20: Observation-delayed RDCs for recurrent PPO

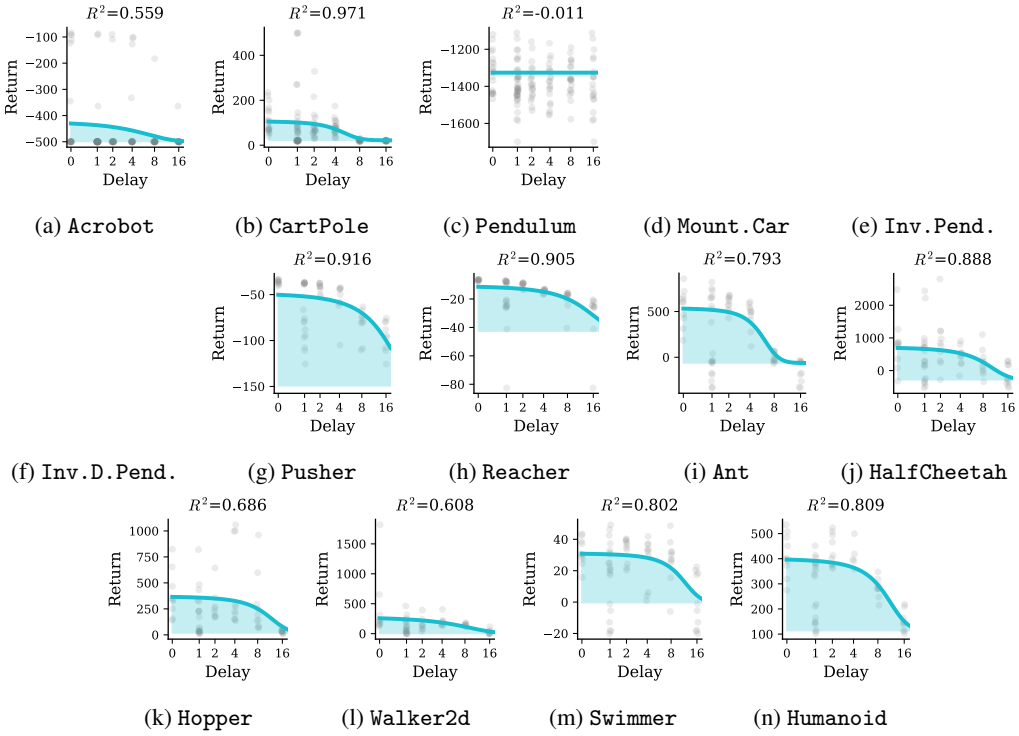
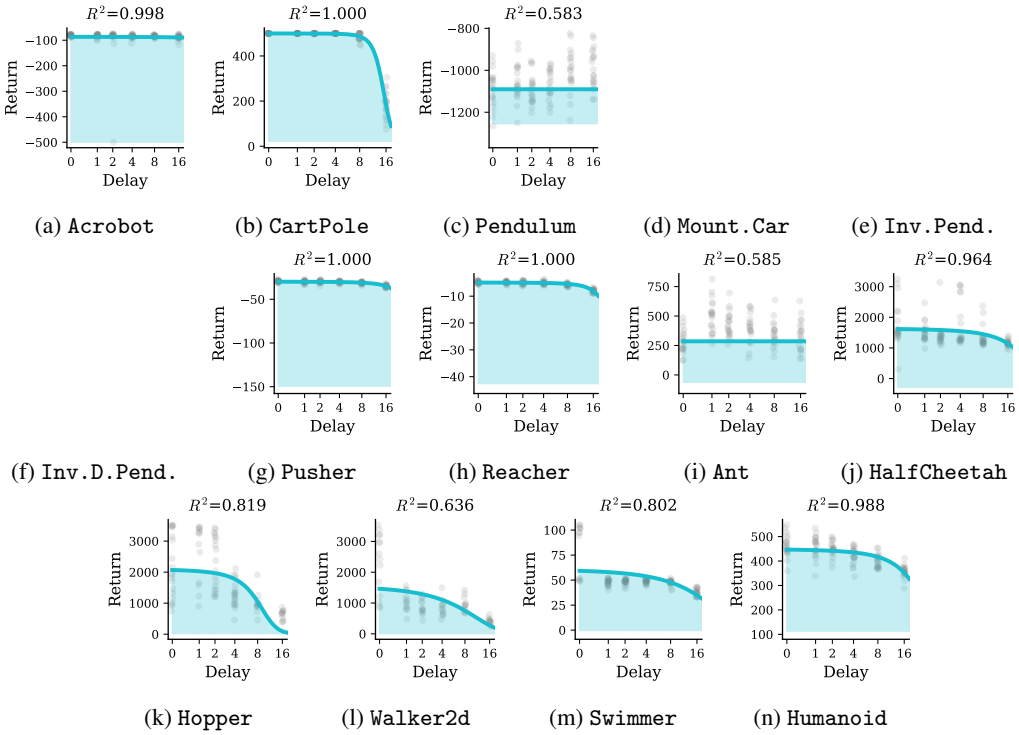


Figure 21: Action-delayed RDCs for recurrent PPO

Forward model PPO:



(a) `Acrobot` (b) `CartPole` (c) `Pendulum` (d) `Mount.Car` (e) `Inv.Pend.`

(f) `Inv.D.Pend.` (g) `Pusher` (h) `Reacher` (i) `Ant` (j) `HalfCheetah`

(k) `Hopper` (l) `Walker2d` (m) `Swimmer` (n) `Humanoid`

Figure 22: Observation-delayed RDCs for recurrent PPO

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: All claims are supported by the presented results.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We discuss limitations in a dedicated "Limitations" subsection of the Discussion.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not include theoretical results.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Details about the experiments are provided in the main text and in the Appendix. The code we provide permits replication of the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The code is open-sourced and available on an annonymous repository; no specific dataset was used in the study.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: We provide settings for hyperparameters and our rationale for choosing them. For specific algorithms being tested we use their published "default" parameters without additional justification.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Statistical tests were performed to assess the significance of Pearson correlations in the Results section.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

Justification: The paper reports the compute resources used for the experiments in Appendix C.1: 28,560 runs were executed on CPU nodes of a university high-performance computing cluster, with each run allocated 4 CPU cores and 12 GB of memory. No GPUs were used.

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: This research does not involve human or animal subjects or sensitive data. We have adhered to all the integrity standards regarding citations of previous work and reproducibility.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: In the Discussion we argue that our work may help to better understand how both robots and biological systems can mitigate the effects of delays. We do not discuss societal impacts of, e.g., improved robot controllers in detail.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not pose such risks.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Our experiments make use of Gymnasium and Stable-Baselines3, two open-source code libraries. We credit the original authors.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: We provide a well-document code repository with detailed instruction for installation, usage and reproducibility.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing nor research with human subjects.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: The core method development in this research does not involve LLMs as any important, original, or non-standard components.